

3D Geometry Processing

Exercise 06: Surface Reconstruction

Submission deadline: 21.04.2026, 13:00

Late submissions are not accepted

Submission rules

You should submit a single .zip compressed file named `Exn-GroupMemberNames.zip` where n is the number of the current exercise sheet. *Please order the names alphabetically by last name to make life easier for us :)* The file should contain:

- In programming exercises: all code files you changed, and **only** the files you changed (headers and source).
- A single file (.txt, .pdf or .md) with the following contents:
 - Names of all team members,
 - Any difficulties encountered and how you resolved them,
 - Optionally any comments about the exercise.
 - Your solutions to the theoretical exercises (if any)
 - (Only in case you used any AI assistants:) List for which purposes/problems you used them, and your judgement of where this was helpful, useless, or misleading.
- (Only in case you used any AI assistants:) Include all your chat transcripts as separate files in an archive `AI-transcripts.zip` inside the main .zip.

1 Practical Exercise (2 pts)

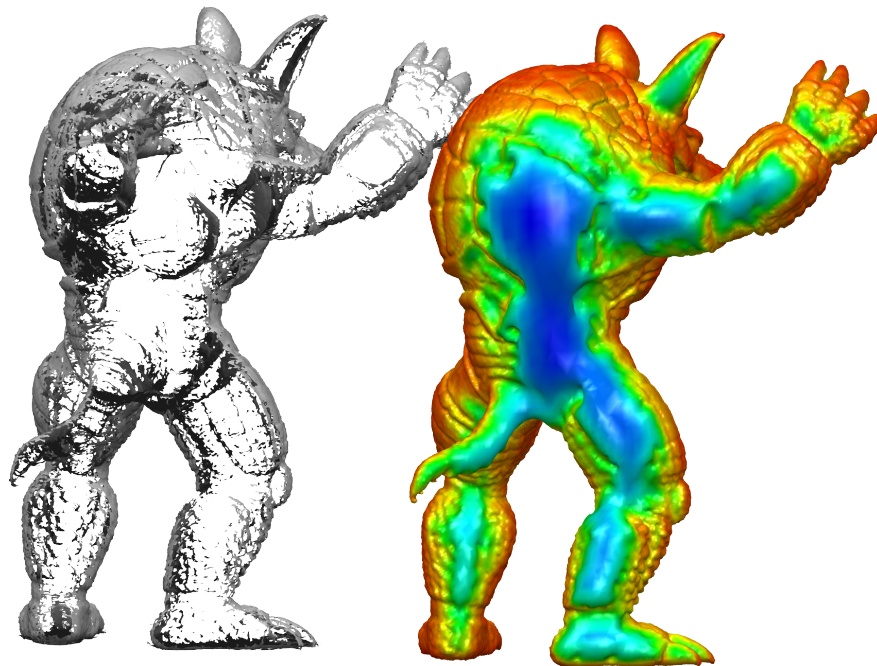


Figure 1: Armadillo Man reconstructed from partial scan *ArmadilloBack.ply* (left), rendered with color-coded point density / reconstruction confidence (right)

The goal of this exercise is to get an idea of how Poisson surface reconstruction [1] works in practice, the effects of some of its variants and parameters, and to get some experience in interfacing different code bases (“plumbing”).

Your first task is to finish integrating the reference implementation¹ into an OpenFlipper plugin.

The input data is an unordered point set with normals, which we represent in OpenFlipper using OpenMesh meshes. We ignore any connectivity (edges, faces) that may exist and only use the vertices and their normals. This means you can also use the plugin directly on any surface mesh.

Plugin code template

The main plugin class already contains code that packs points and normals from various OpenFlipper data types into a minimalistic data structure, a flat `std::vector<double>`, where groups of 6 consecutive values represent one point (3 for position, 3 for normal). You don’t have to change anything here.

The interface to PoissonRecon is declared in `PoissonReconstructionT.hh` and implemented in `PoissonReconstructionT_impl.hh`. It may be helpful to read the section “HEADER-ONLY LI-

¹<https://github.com/mkazhdan/PoissonRecon>

BRARY” of the PoissonRecon library’s readme², which describes the stream interface we’ll interface with, as well as the example code³, which was the basis for implementing our interface.

Coding Task (1 pt)

- Download and unzip `Plugin-Ex06-PoissonReconstruction.zip`.
- Read the code in its top-level folder (`PoissonRecon/` is too much to read)
- **Fill in the TODOs in** `PoissonReconstructionT_impl.hh` to pass data to and from the PoissonRecon library. *Tip:* To debug input and output streams separately, you can (temporarily) use the input stream from the example which generates random sphere samples.
- Test on a few inputs to verify everything works.

The C++ code in this exercise heavily uses templates. We hope this will not be too confusing for the C++-inexperienced among you. If you run into any difficulties or crashes, please feel free to reach out to us via Ilias Forum, email or in person, we don’t want you to get stuck here.

Experiments and research (1pt)

After you have finished implementing the plugin, it’s now time to experiment a bit. Try it out on various inputs (see below for data sources) and using various settings, and report in your submission what you found out about the effect of the configurable parameters.

The plugin exposes what we believe to be the most important / interesting parameters. The most important parameter is the maximum octree depth. We have also exposed two more parameters in the UI, one to select *SSD surface reconstruction* [2], and one to test *Screened Poisson surface reconstruction* [3].

Optional: Try out more of the parameters that PoissonRecon allows to set, either by hard-coding them, or preferably extending the UI to expose them to the user and makes it easy for yourself to compare results.

Try to **answer the following questions** (a sentence or two for each is enough, you don’t have to go super deep):

1. What are the most important effects of the octree depth setting? What’s a “good” setting for the inputs you tested?
2. What is the meaning of the “weight” output property (used for color-coding the output)? How might one want to use it?
3. *Optional:* What is the rough idea behind screened Poisson surface reconstruction? Demonstrate the effect of the point weight on an example using screenshots.

²<https://github.com/mkazhdan/PoissonRecon/blob/cd6dc7d3/README.md>

³<https://github.com/mkazhdan/PoissonRecon/blob/cd6dc7d3/Src/Reconstruction.example.cpp>

4. *Optional*: What is the rough idea behind SSD reconstruction? Can you find cases where it is preferable over classic Poisson reconstruction?
5. *Optional*: Can you find or create an input that looks reasonable (to a non-expert) but results in a bad reconstruction?

If you make any interesting observations while experimenting, do write them down.

Input data sources

We provide some sample data in Ilias that you can process, but feel free to explore other data sources.

We uploaded some real scan data based on⁴ raw data from the *Stanford 3D Scanning Repository*⁵, which contains some of the most iconic meshes you'll encounter in the geometry processing and computer graphics literature. Try reconstructing from just a subset of inputs (e.g. a single range scan), or from all of them together (you have to mark multiple inputs as *target*).

You can also use meshes we have previously supplied, your own meshes created using photogrammetry⁶, or any interesting data you can find online or generate yourself.

There are some data sources linked in the PoissonRecon readme as well⁷.

Tip: if your computational resources are limited but you want to experiment with deeper octree depths on real inputs without waiting forever or running out of RAM, try to cut an input down to a more manageable size by selecting and deleting vertices.

References

- [1] Michael M. Kazhdan, Matthew Bolitho, and Hugues Hoppe. "Poisson surface reconstruction". In: *Proceedings of the Fourth Eurographics Symposium on Geometry Processing, Cagliari, Sardinia, Italy, June 26-28, 2006*. Eurographics Association, 2006, pp. 61–70. DOI: [10.2312/SGP/SGP06/061-070](https://doi.org/10.2312/SGP/SGP06/061-070).
- [2] Fatih Calakli and Gabriel Taubin. "SSD: Smooth Signed Distance Surface Reconstruction". In: *Comput. Graph. Forum* 30.7 (2011), pp. 1993–2002. DOI: [10.1111/J.1467-8659.2011.02058.X](https://doi.org/10.1111/J.1467-8659.2011.02058.X).
- [3] Michael M. Kazhdan and Hugues Hoppe. "Screened poisson surface reconstruction". In: *ACM Trans. Graph.* 32.3 (2013), 29:1–29:13. DOI: [10.1145/2487228.2487237](https://doi.org/10.1145/2487228.2487237).

⁴We aligned the raw range scan data using the given registration data (.conf files) and merged them per-setup into single OpenFlipper-compatible .ply files

⁵<https://graphics.stanford.edu/data/3Dscanrep/>

⁶If you happen to have exported a raw point cloud from your data while your trial license was valid, that's perfect! You can also use the mesh, but it's less realistic as input data (cf. the info box "Range data versus 3D models - a caveat on the use of these models" on the 3Dscanrep website. Sorry, we didn't consider trial license expiry in time to ask you for this export.

⁷e.g. <https://www.cs.jhu.edu/~misha/Code/PoissonRecon/eagle.points.ply> or <https://www.cs.jhu.edu/~misha/Code/PoissonRecon/torso.zip>